

Eightfold Candidate Profile Pipeline — Design Document

Candidate Name: John Doe | Email: john.doe@example.com | Target: Eightfold Candidate Pipeline

1. Pipeline Breakdown

The system processes multiple recruitment sources end-to-end through a modular sequence: **detect** (identifies source type from file extensions/URLs) → **extract** (extracts raw key-value pairs into intermediate dicts) → **normalize** (standardizes E.164 phones, YYYY-MM dates, ISO-3166 countries, and skills) → **merge** (groups candidates via match-key blocking, resolves conflicts, and generates provenance) → **confidence** (assigns scores using source weight + consensus bonus) → **project** (filters/remaps fields via runtime config) → **validate** (validates output against custom JSON Schema).

2. Canonical Schema & Normalized Formats

The internal record uses a strict Pydantic model representation with normalized fields:

- **Phones:** Standardized to E.164 (e.g. +91 99999 99999) using the *phonenumbers* library, falling back to IN (+91) region.
- **Dates:** Standardized to YYYY-MM format; free-text (e.g., 'Jan 2021-Present') parsed using *python-dateutil* and regex; 'Present' maps to null end date.
- **Country:** Normalized to ISO-3166 alpha-2 (e.g., 'US', 'IN') via dictionary lookup of common names and country aliases.
- **Skills:** Case-insensitive matching against a canonical skills list with alias expansion (e.g. 'ReactJS' → 'React').

3. Merge & Conflict-Resolution Policy

Match Keys: Identity grouping uses a blocking key chain: exact match on normalized email, exact match on normalized phone, or fuzzy match on full name (Levenshtein distance > 0.85). **Conflict Resolution:** Field-level winners are determined by configurable source priority order: for identity and contact details, *resume* > *ATS JSON* > *recruiter CSV* > *notes* > *GitHub*; for skills and links, *GitHub* > *resume* > *ATS JSON* > *CSV*; for experience/education, *resume* > *ATS JSON* > *CSV*. **Confidence Formula:** Each field confidence is computed as: $base_score(extraction_method) + corroboration_bonus - recency_decay$. Base scores are: structured = 0.9, GitHub API = 0.8, resume regex = 0.6, notes = 0.4. Corroboration adds +0.05 per agreeing source, capped at 1.0.

4. Runtime Config Handling

The projection layer maps the canonical record to custom output shapes at runtime. Config defines: **path** (output path, supporting dot-notation), **from** (canonical source path), **type** (expected type), and **on_missing** policy (*null* keeps key as null; *omit* drops key; *error* collects field-level error and raises exception). The system generates a dynamic JSON Schema matching the config shape and validates the projected output using *jsonschema*. Errors are aggregated per-candidate.

5. Edge Cases Handled

- **1. Multiple Emails:** If same candidate has different emails, they remain separate candidates unless name/phone matches; noted as a known limitation.
- **2. Missing Country Code:** Assumes regional fallback (IN) but scales field confidence down by a factor of 0.5.
- **3. Source Failure:** Malformed CSV lines or GitHub API rate-limits/404s are logged as warnings and skipped; the pipeline continues processing other rows.
- **4. Conflicting Experience:** Takes highest-confidence source for *years_experience* (e.g. resume = 3, notes = 5 → output 3) and stores the other in provenance for audit.

6. Deliberately Left Out (Future Enhancements)

- **LinkedIn Scraping:** Omitted due to TOS restrictions/auth requirements; mock ATS JSON acts as a placeholder for manual data uploads.
- **Fuzzy Name Clustering:** Advanced cross-source clustering (e.g. Jaro-Winkler/double-metaphone) omitted for a simple Levenshtein implementation.
- **User Interface:** UI polish skipped in favor of a robust, deterministic, testable CLI tool outputs JSON to standard output and files.